

Memory Map

Address Range	Peripheral	Registers
0x00000000 - 0x00007FFF	Block RAM	Instructions + Data Memory (32 KB)
0x10000000	UART	<u>TX_DATA</u> : write byte to transmit
0x10000004	UART	<u>RX_DATA</u> : read received byte
0x10000008	UART	<u>STATUS</u> : bit 0 = TX busy, bit 1 = RX Valid
0x1000000C	UART	<u>CONTROL</u> : bit 0 = RX Interrupt enable
0x10001000	GPIO	<u>DIR</u> : 1 per pin = 1 = output
0x10001000	GPIO	<u>OUT</u> : write output values
0x10001000	GPIO	<u>IN</u> : read input values (value-only)
0x10002000	PWM	<u>PERIOD</u> : Counter top Value
0x10002004	PWM	<u>DUTY</u> : Compare Value
0x10002008	PWM	<u>CONTROL</u> : Bit 0 = enable
0x10003000	Timer	<u>COUNT</u> : current counter value (read-only)
0x10003004	Timer	<u>COMPARE</u> : value that triggers interrupt
0x10003008	Timer	<u>CONTROL</u> : bit = 0, enable , bit 1 = interrupt enable
0x1000300C	Timer	<u>PRESCALER</u> : clock divider value

- Memory map needed for organizing addresses to different components,
- Acts as a guide for where the system to know where code runs, where variables are stored and how to communicate between hardware
-

Block Diagram

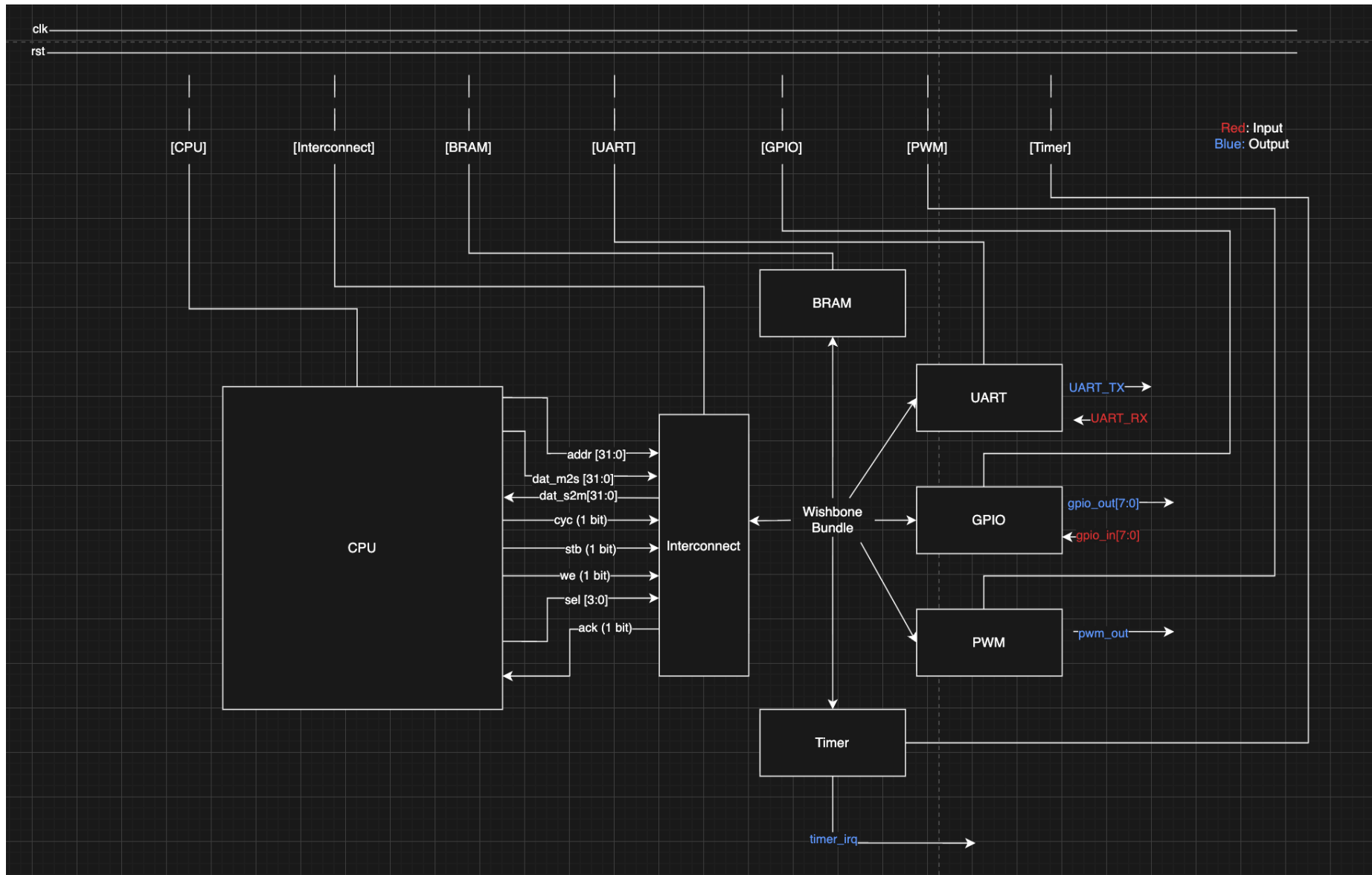


Figure 1: Top-Level SoC Block Diagram

Figure 1 Explanation:

What the diagram shows:

- Shows every major hardware block and how they are all connected through the wishbone bus.
- The diagram shows 7 top-level blocks: the CPU, Wishbone interconnect, Block RAM (BRAM), GPIO, UART, PWM and the timer. (Do note that there is a global clock and reset going to every block in the diagram)

The CPU Block:

- The CPU is the only bus master of the system. This means that the cpu is the only one that can initiate transactions through the wishbone bus everything else is only a slave that responds when addressed by the CPU
- CPU is a 3 stage pipeline, this diagram only shows the Wishbone master ports that drives it outwards

Signal	Description
Wb_addr[31:0]	It is a 32 bit address that the cpu reads from or writes to. This is how the interconnect decides which peripheral it sends the transaction to.
wb_dat_m2s[31:0]	Master-to-slave data. 32 bit write data sent from the CPU to the Peripheral
Wb_dat_s2m[31:0]	Slave-to-Master data. 32 bit read data sent from the peripheral to the CPU
Wb_cyc (1 bit)	Cycle valid. Held high the entire duration of a bus transaction to indicate that i am currently using this bus
Wb_stb (1 bit)	Strobe. Pulsed high to indicate this specific clock cycle is an active request . always used together with cyc
Wb_we (1 bit)	Write enable. 1 means writing and 0 means reading
Wb_sel[3:0]	Byte select, each bit corresponds to 1 byte lane of the 32 bit data bus . for a full 32 bit word write all 4 bits are high (0b1111)
Wb_ack (1 bit)	For the peripheral to say they acknowledged when they receive data one pulse means data is complete

The Wishbone Interconnect Block

- Interconnect is a passive routing block. Has no memory or logic beyond address comparison. The job of the wishbone interconnect block is to check the address and send the data accordingly to which peripheral it belongs to (on every clock cycle)
- Two stage address decode works as follows: bits [31:28] are checked if 0x0 then it goes to BRAM (Block RAM) if they are 0x1 then there is a second check that goes on bits [15:12] routes to correct peripheral as shown

Addr[31:28]	addr[15:12]	Selected Peripheral
0x0	-	Block RAM (0x00000000 - 0x00007FFF)
0x1	0x0	UART (0x10000000)
0x1	0x1	GPIO (0x10001000)
0x1	0x2	PWM (0x10002000)
0x1	0x3	Timer (0x10003000)

Peripheral Blocks

- Each peripheral block connects to the Shared Wishbone bus Bundle and also each peripheral has external i/o pins connecting to the physical world

Signal	Description
BRAM	Store both instruction and data. CPU fetches instructions from this and also stores/loads data from here during mem/writeback stage
UART	Serial communication peripheral, UART_TX is an output pin that sends data out, UART_RX is an input pin that receives data in. It communicates with 115200 BAUD, 8N1
GPIO	General purpose input and output, gpio_out [7:0] drives 8 output pins, gpio_in [7:0] reads 8 input pins direction is set per pin via a direction register

PWM	Pulse width modulation output Generates a square wave that changes on frequency depending on a configurable duty cycle
Timer	Free running 32 bit counter, Generates an interrupt to the cpu depending on the user choice timing / when it reaches a programmed compare value

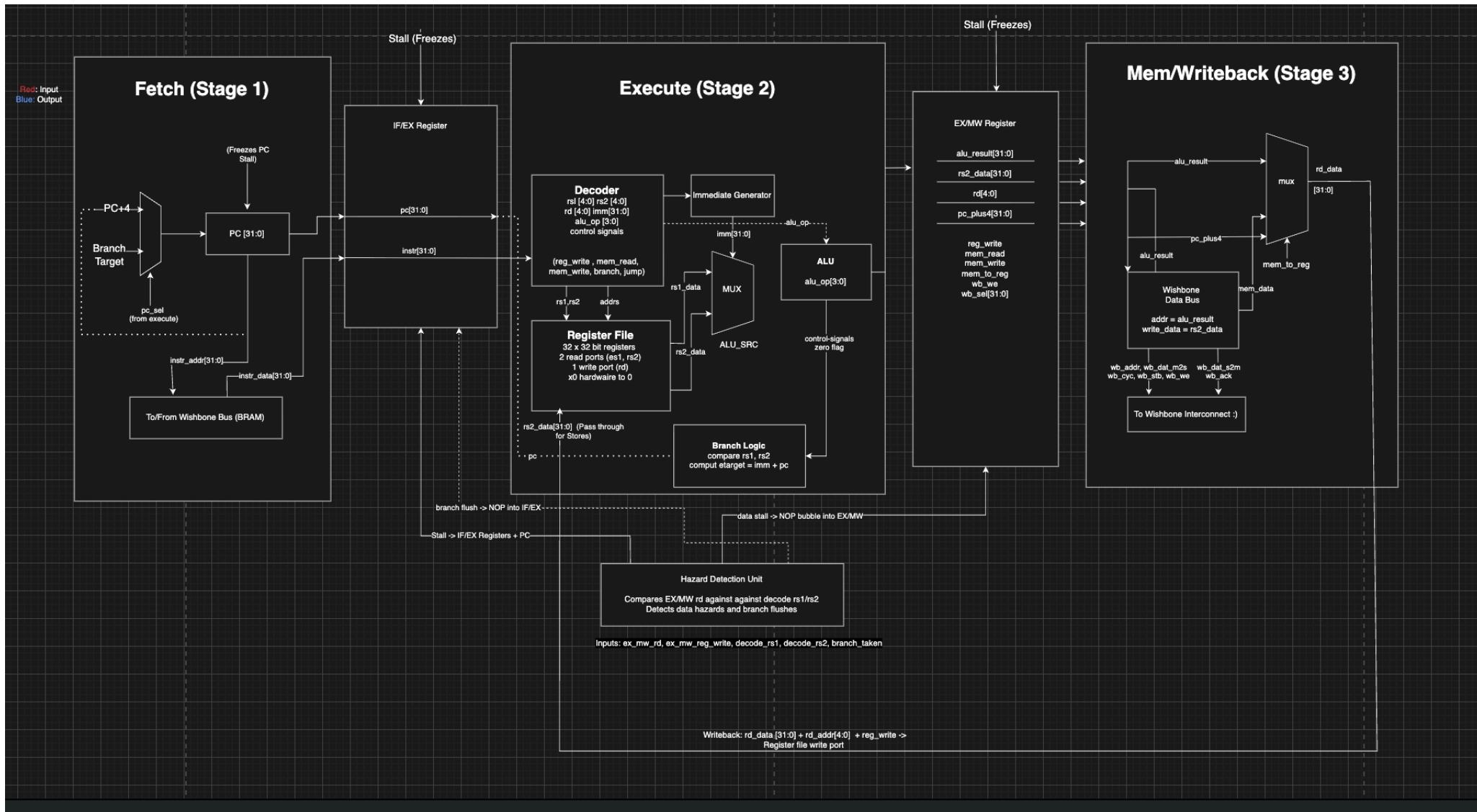


Figure 2: CPU Pipeline Internal Diagram

Figure 2 Explanation:

What the diagram shows:

- This diagram focuses mostly inside the CPU from figure 1, reveals the 3 pipeline stages of the cpu; fetch ,execute and mem/writeback.
- Between these 3 different stages are pipeline registers that latch (hold) the values computed by one stage of the cpu and hold them until the next cycle
- The different stages work like a factory where all stages are constantly working there is not a single one not working.

Stage 1 (Fetch):

- It's only job is to get the next instruction from memory, also holds the PC (Program Counter) which is a 32 bit register holding the next instruction's address to fetch next.

PC Update Logic

- On every normal clock cycle the PC will jump by 4 since every address is 32 bits long it jumps 4 bytes, however there are 2 different cases that can override this normal case
 - Branch taken in execute: the branch logic block computes a branch target address ($PC + \text{Immediate offset}$) then drives it back to fetch stage via pc_sel signal. When pc_sel is asserted it goes somewhere else other than $PC+4$
 - A pipeline stall: if hazard detection unit asserts stall pc register is frozen and no new fetch happens

Instruction Memory Access

- Fetch stage drives instr_addr[31:0] to Wishbone bus pointing to BRAM. One clock cycle later instr_data[31:0] comes back containing 32 bit instruction word the instruction word is then latched into IF/EX register along with current PC value

IF/EX Pipeline Register Contents

- PC[31:0] : address of the instruction just fetched, needed later for branch target calculation ($pc + imm$) and for $pc + 4$ writeback on JAL/JALR
- INSTR[31:0]: raw 32-bit instruction word from BRAM, decoder in execute reads this

Stage 2 (Execute):

- This stage decodes instruction, reads the register file and computes the ALU result, perform branch comparison and detects hazard

Decoder

- Decoder reads raw instr[31:0] from the IF/EX register and extracts all fields for RISC V the opcode is always in bits [6:0] the destination register is in bits [11:7] and source registers are in bits [19:15] and [24:20]. The decoder produces the control signal that steer every mux and enable signal downstream

Signal	Description
Rs1[4:0] , rs2[4:0]	Source register address, used to index into the 32 entry register file
Rd[4:0]	Destination register address, passed forward to ex/mw register file so writeback knows where to write result
Imm[31:0]	Sign extended immediate value. Assembled by immediate generator from scattered bits in instruction encoding
Alu_op [3:0]	Tells ALU which operation to perform
Mem_read mem_write	High when instruction is a load or store
Reg_write	High when instruction writes a result back to register file
Branch_jump	Flags telling when whether to evaluate branch condition or perform an unconditional jump

Register File

- The register file holds all 32 general purpose registers, each 32 bits wide it has two read ports and one write port so it can supply both rs1 and rs2 data in the same clock cycle while simultaneously accepting a writeback from stage 3. Register x0 is hardwired to zero any write is silently discarded as it always reads 0x00000000.

ALU and ALU_SRC Mux

- ALU_SRC mux selects whether the second ALU input comes from rs2_data (register-register instructions like ADD, AND) or from imm[31:0]. The ALU computes the result and also produces a zero flag used by the branch logic

Branch Logic

- For Branch instructions the branch logic compares rs1 and rs2 using the ALU zero flag and computes target address from as PC + imm. If branch is taken it sends the branch target back to the PC in stage 1 and asserts branch_flush to inject a NOP into the IF/EX register, disregarding the incorrectly-fetched instruction

EX/MW Pipeline Register Contents

Signal	Description
ALU_result [31:0]	The computed ALU output, For arithmetic / logic instructions this is the final result for loads and stores this is the memory address
Rs2_data [31:0]	The value of the rs2 registers. Passed forward to stage 3 so it can use write data from store (SW) instructions
Rd[4:0]	Destination register address. Forwarded to writeback
Pc_plus4[31:0]	PC + 4 forwarded to JAL/JALR can write the return address
Reg_write, mem_read, mem_write, mem_to_reg, wb_sel[3:0]	Controls signals from the decoder forwarded to stage 3

Stage 3: Mem/ Writeback

- This stage handles two things performing data memory transactions (loads and stores) and writing the final result back to the register file

Wishbone Data Bus

- When instruction is a load or store Stage 3 drives the wishbone bus towards the interconnect, The address is alu_result (address computed by the ALU in stage 2) and the write data is rs2_data for stores. The peripheral returns wb_dat_s2m with the loaded value on an ack

Writeback Mux

- The mem_to_reg control signal selects what gets written back to the register file. For ALU instructions it selects alu_result. For Load instructions it selects the data read from memory for jumps it selects pc_plus4 the selected value goes to rd_data [31:0] and is written to the register file at the rd address.

Hazard Detection Unit

- This sits below all 3 stages and monitors for two types of pipeline hazards.

- Data hazard: occur when an instruction in execution needs a register value that the previous instruction has not written back yet. The HDU detects this by comparing the rd address in the EX/MW register against the rs1/rs2 addresses being decoded . When a hazard is detected it stalls the PC and IF/EX register and injects a NOP bubble into EX/MW.
- Control Hazards: when a branch is taken the instruction that was taken by IF/EX is wrong The HDU inserts a NOP so that wrong instruction never executes

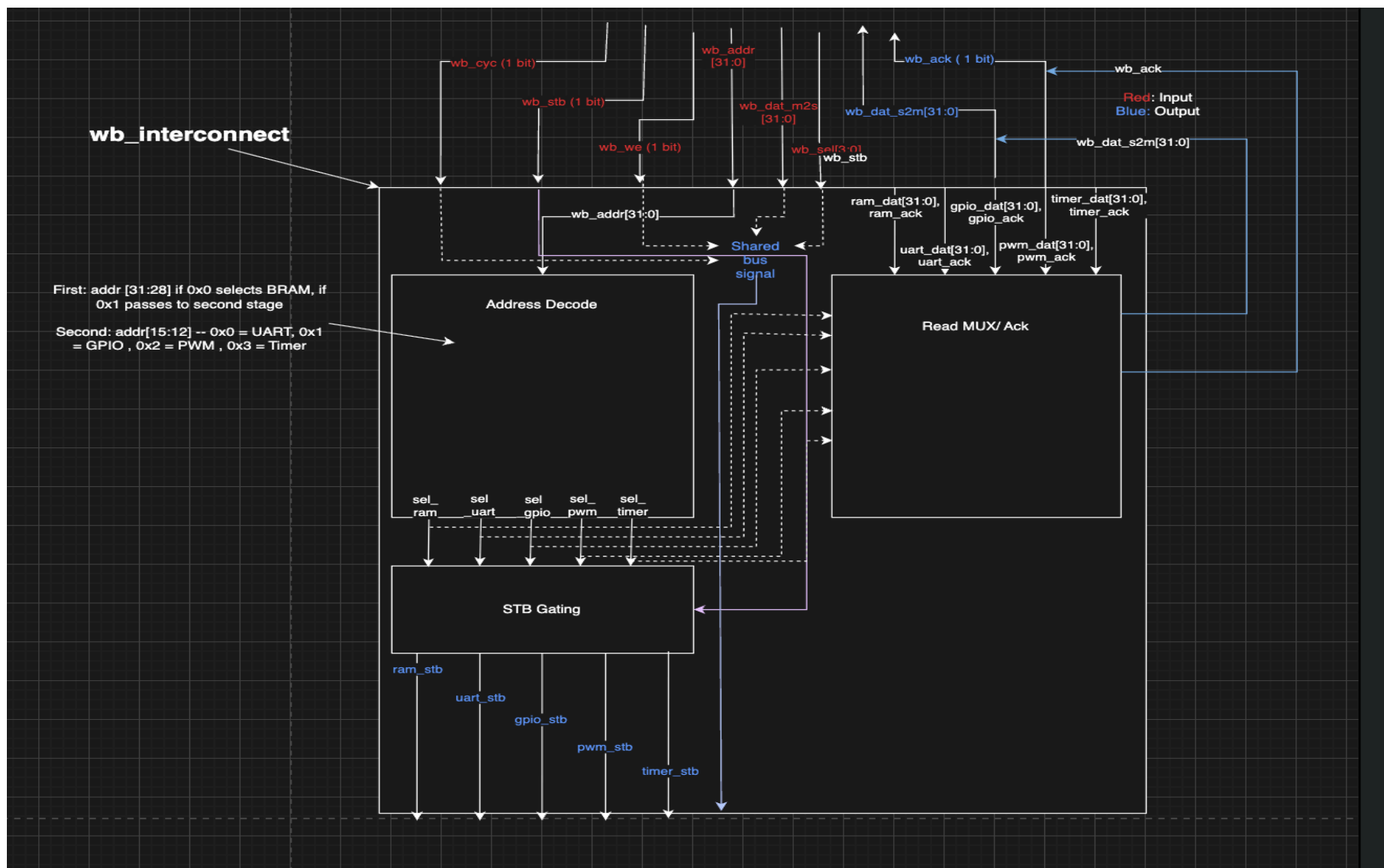


Figure 3: Wishbone Interconnect Detail

Figure 3 Explanation:

What this Diagram shows:

- Zooms inside interconnect block from diagram 1, shows how address decode logic and STB gating work
- Takes the single CPU bus transactions and routes it to exactly one peripheral, on the way back , it collects the peripheral's response and returns it to the CPU all logic is purely combinational except for STB gating

Diagram explanation

- Module has three sections in the diagram
 - Address decode block: reads wb_addr and produces sel_signal
 - STB gating block: multiplies sel_signal with wb_stb to produce per peripheral stb lines
 - Read MUX / Ack block: Selects which peripheral reads data and ack to pass back to the CPU

Input from the CPU

Signal	Width	Description
Wb_cyc	1 bit	Bus cycle is active. Interconnect only processes transactions while this is asserted
wb_stb	1 bit	Valid transaction strobe. Combined with cyc to qualify with a bus access
Wb_addr[31:0]	32 bits	Full 32 bit address. The interconnect inspects specific bit ranges to determine the target peripheral
wb_dat_m2s[31:0]	32 bits	Write data. Broadcasted to all peripherals but only the selected one latches it
wb_we	1 bit	Write enable. Broadcast to all peripherals
Wb_sel[3:0]	4 bit	Byte Select. Broadcast to all peripherals

The Address Decode Block

- The Address Decode block inspects specific bits of `wb_addr[31:0]` to decide which peripheral owns this transaction. It runs a two-stage check: first it looks at `addr[31:28]`, if that nibble is `0x0` it selects BRAM immediately.
- if it is `0x1` it moves to the second stage and checks `addr[15:12]` to distinguish between UART (`0x0`), GPIO (`0x1`), PWM (`0x2`), and Timer (`0x3`).
- The result is five one-bit select signals, `sel_ram`, `sel_uart`, `sel_gpio`, `sel_pwm`, `sel_timer`, exactly one of which is high at any time. All five are purely combinational assign statements; there is no clock involved.

The STB Gating Block

- The STB Gating block takes each `sel_` signal and ANDs it with `wb_stb` to produce a per-peripheral strobe: `ram_stb = wb_stb and sel_ram`, and so on for each peripheral.
- This ensures a peripheral only reacts when the CPU is actually requesting a transfer, not just whenever its address range happens to appear on the bus.
- These gated strobes (`ram_stb`, `uart_stb`, `gpio_stb`, `pwm_stb`, `timer_stb`) are the outputs you see at the bottom of Diagram 3, and they connect directly to the `wb_stb` port of each peripheral module.

The Read MUX / Ack Block

- Every peripheral has its own private read-data wire (`ram_dat`, `uart_dat`, etc.) and its own acknowledge wire (`ram_ack`, `uart_ack`, etc.).
- The Read MUX picks the correct one using the `sel_` signals and presents it on the single `wb_dat_s2m` and `wb_ack` wires that the CPU reads. For the data mux this is a priority chain of ternary operators.
- for the ack it is an OR of each `sel_` and `peripheral_ack` term. Because only one `sel_` is ever high, only one peripheral's data and ack can ever appear on the output at a time.

Complete Transaction Walkthrough

- When the CPU executes a load or store, it puts the target address on `wb_addr`, asserts `wb_cyc` and `wb_stb`, and sets `wb_we` to indicate read or write.
- The Address Decode block immediately produces the correct `sel_` signal. The STB Gating block gates `wb_stb` through to the matching peripheral's strobe, leaving all other peripheral strobes at zero.

- The selected peripheral sees its strobe go high, performs the operation (reading or writing its internal register), drives its data onto its private dat wire, and asserts its ack for one cycle.
- The Read MUX forwards that data and ack back to the CPU on wb_dat_s2m and wb_ack.
- The CPU sees wb_ack go high, captures the data, and drops wb_cyc and wb_stb. The entire transaction takes one clock cycle for simple peripherals.

Why This Design Is Simple

- The entire wb_interconnect module contains no registers and no clock. It is roughly 10-15 lines of assign statements.
- Because it is purely combinational, the routing decision resolves in the same clock cycle the CPU presents the address, there is zero added latency.
- Adding a new peripheral only requires one new sel_ assign, one new STB gate, and one new line in each mux. In C++ terms this is equivalent to a const lookup table: no side effects, no state, and completely deterministic for any given input address.

Timing Diagram

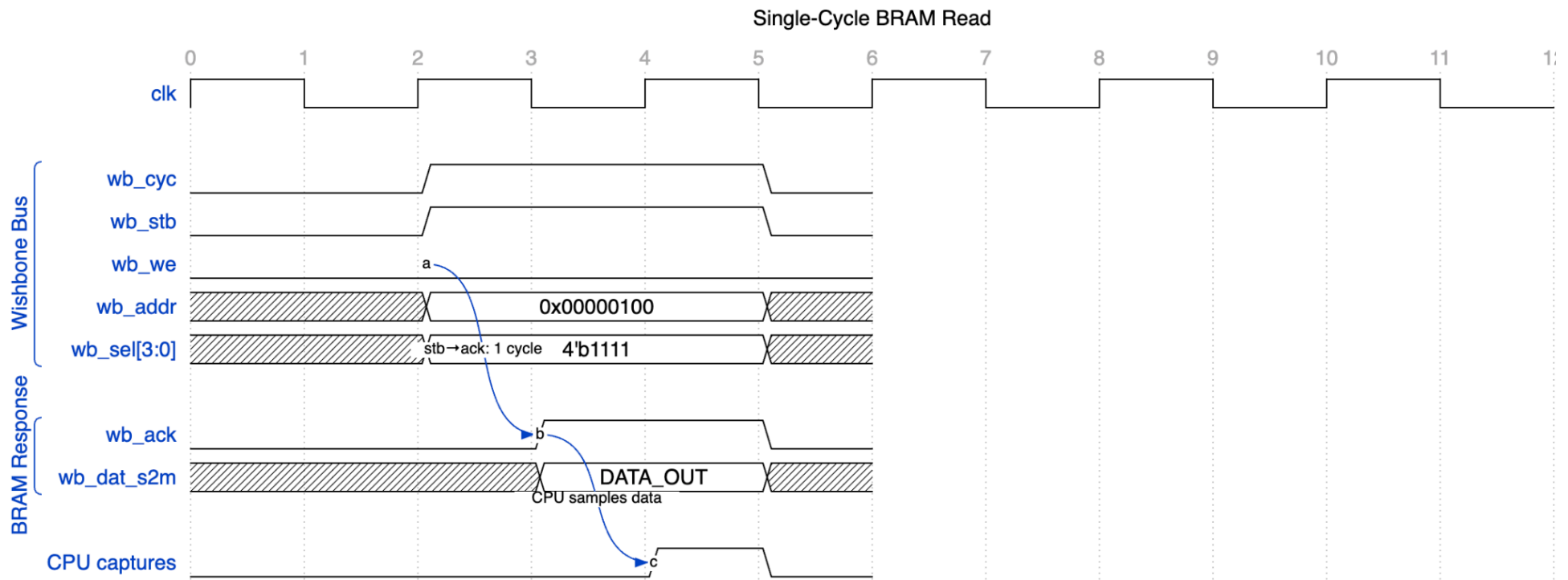


Figure 1: Single Cycle BRAM Read

Code:

```
{ signal: [
  { name: "clk",    wave: "p.....", period: 2 },
  {},
  ["Wishbone Bus",
    { name: "wb_cyc",  wave: "0.1..0" },
    { name: "wb_stb",  wave: "0.1..0" },
    { name: "wb_we",   wave: "0.....",   node: "..a.." },
    { name: "wb_addr", wave: "x..=..x", data: ["0x00000100"] },
    { name: "wb_sel[3:0]", wave: "x..=..x", data: ["4'b1111"] }
  ],
  {},
  ["BRAM Response",
    { name: "wb_ack",  wave: "0..1.0",   node: "...b.." },
    { name: "wb_dat_s2m", wave: "x..=..x", data: ["DATA_OUT"] }
  ],
  {},
  { name: "CPU captures", wave: "0...10",   node: "....c." }
],
edge: [
  "a~>b stb→ack: 1 cycle",
  "b~>c CPU samples data"
],
head: {
  text: "Single-Cycle BRAM Read",
  tick: 0
},
config: { hscale: 2 }
}
```

Explanation:

Block diagram reference: Top-Level SoC -> CPU -> wb_interconnect -> wb_ram

CPU reads 32-bit word from BRAM at address 0x00000100. Stb asserts cycle 2, ack returns cycle 3, CPU captures data on cycle 4 rising edge.

What's happening cycle by cycle:

- Cycle 1: Bus idle, everything low/unknown
- Cycle 2: CPU drives cyc, stb HIGH. Address valid. we stays LOW (it's a read)
- Cycle 3: BRAM responds — ack goes HIGH, dat_s2m becomes valid
- Cycle 4: Rising edge — CPU captures dat_s2m. Sees ack → deasserts cyc/stb
- Cycle 5: Bus idle again

Cycle 2: CPU drives wb_cyc and wb_stb HIGH, places address 0x00000100 on the bus, keeps wb_we LOW (read).

Cycle 3: BRAM sees stb, looks up address in one clock — wb_ack goes HIGH and dat_s2m becomes valid.

Cycle 4: On the rising edge, CPU samples dat_s2m (captures data), sees ack HIGH → deasserts cyc/stb. Transaction complete.

The BRAM is a single-cycle slave: it responds with ack one clock after stb is asserted. The interconnect simply passes signals through based on the address decode logic from your Wishbone block diagram.

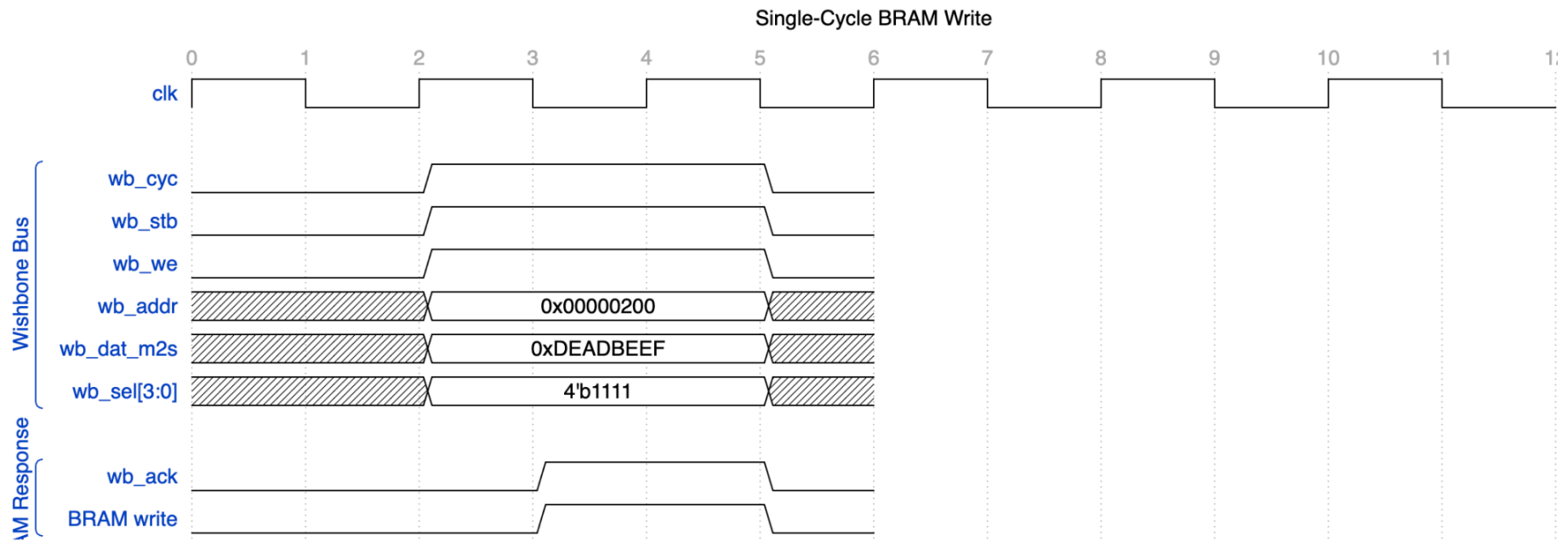


Figure 2: Single Cycle BRAM Write

Code:

```
{ signal: [
  { name: "clk",    wave: "p.....", period: 2 },
  {},
  ["Wishbone Bus",
    { name: "wb_cyc",  wave: "0.1..0" },
    { name: "wb_stb",  wave: "0.1..0" },
    { name: "wb_we",   wave: "0.1..0" },
    { name: "wb_addr", wave: "x.=.x", data: ["0x00000200"] },
    { name: "wb_dat_m2s", wave: "x.=.x", data: ["0xDEADBEEF"] },
    { name: "wb_sel[3:0]", wave: "x.=.x", data: ["4'b1111"] }
  ],
  {},
  ["BRAM Response",
    { name: "wb_ack",  wave: "0..1.0" },
    { name: "BRAM write", wave: "0..1.0" }
  ]
],
head: {
  text: "Single-Cycle BRAM Write",
  tick: 0
},
config: { hscale: 2 }
}
```

Explanation:

Block diagram reference: Top-Level SoC -> CPU -> wb_interconnect -> wb_ram

Identical timing to read, but wb_we is HIGH and CPU drives dat_m2s.

Key difference from read: wb_we is HIGH, and the CPU drives wb_dat_m2s instead of reading wb_dat_s2m. The BRAM latches the data on the same cycle it asserts ack.

Cycle 2: CPU asserts wb_cyc, wb_stb, and wb_we = HIGH. It also drives the write data on dat_m2s and address on wb_addr. wb_sel = 4'b1111 means all 4 bytes are written.

Cycle 3: BRAM sees stb + we, latches the data into the addressed location. Asserts wb_ack.

Cycle 4 : CPU sees ack on the rising edge -> deasserts all bus signals. Write complete.

Identical timing to a read — the only difference is wb_we is HIGH and the CPU drives dat_m2s instead of reading dat_s2m.

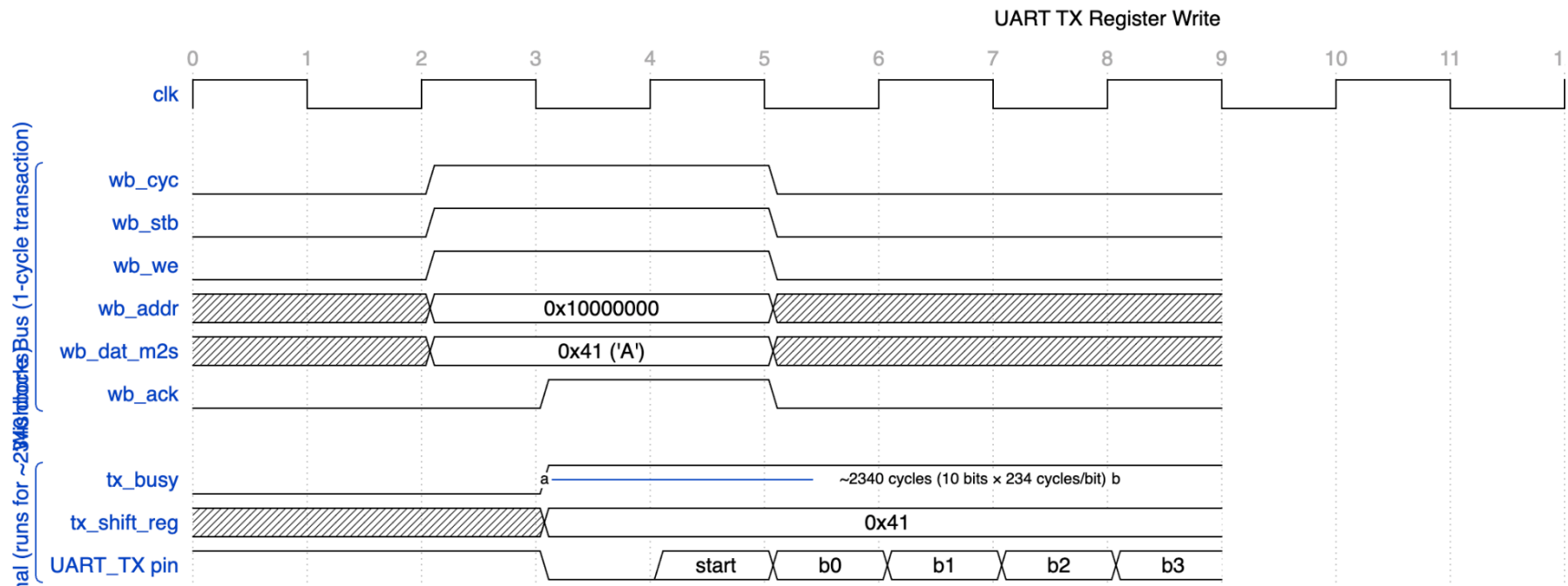


Figure 3: UART TX Register Write

Code:

```
{ signal: [
  { name: "clk",      wave: "p.....", period: 2 },
  {},
  ["Wishbone Bus (1-cycle transaction)",
   { name: "wb_cyc",   wave: "0.1..0..." },
   { name: "wb_stb",   wave: "0.1..0..." },
   { name: "wb_we",    wave: "0.1..0..." },
   { name: "wb_addr",  wave: "x..x...", data: ["0x10000000"] },
   { name: "wb_dat_m2s", wave: "x..x...", data: ["0x41 ('A')"] },
   { name: "wb_ack",   wave: "0..1.0..." }
  ],
  {},
  ["UART Internal (runs for ~2340 clocks)",
   { name: "tx_busy",   wave: "0..1.....", node: "...a....b" },
   { name: "tx_shift_reg", wave: "x..=.....", data: ["0x41"] },
   { name: "UART_TX pin", wave: "1..0=====", data: ["start", "b0", "b1", "b2", "b3"] }
  ]
},
edge: [
  "a~>b ~2340 cycles (10 bits × 234 cycles/bit)"
],
head: {
  text: "UART TX Register Write",
  tick: 0
},
config: { hscale: 2 }
}
```

Explanation:

Block diagram reference: Top-Level SoC -> CPU -> wb_interconnect -> wb_uart

Bus transaction completes in 1 cycle (same as BRAM), but the UART then spends ~2340 clock cycles serializing the byte at 115200 baud.

The Wishbone transaction completes fast, the interconnect routes to UART because $\text{addr}[31:28] = 0x1$ and $\text{addr}[15:12] = 0x0$. But the UART's internal shift register takes 2340 clocks to serialize the byte. Firmware MUST poll the STATUS register before writing the next byte.

Bus timing is identical to BRAM write, the UART register write completes in 1 cycle with ack. The interconnect's address decode routes this to the UART because $\text{addr}[31:28] = 0x1$ and $\text{addr}[15:12] = 0x0$.

But the physical transmission takes much longer: the UART latches the byte into its shift register and begins serializing it at 115200 baud. That's ~2340 clock cycles ($234 \text{ cycles/bit} \times 10 \text{ bits per frame}$). During this time, tx_busy stays HIGH. If the CPU writes another byte before tx_busy clears, the old data would be overwritten — firmware must poll the STATUS register first.

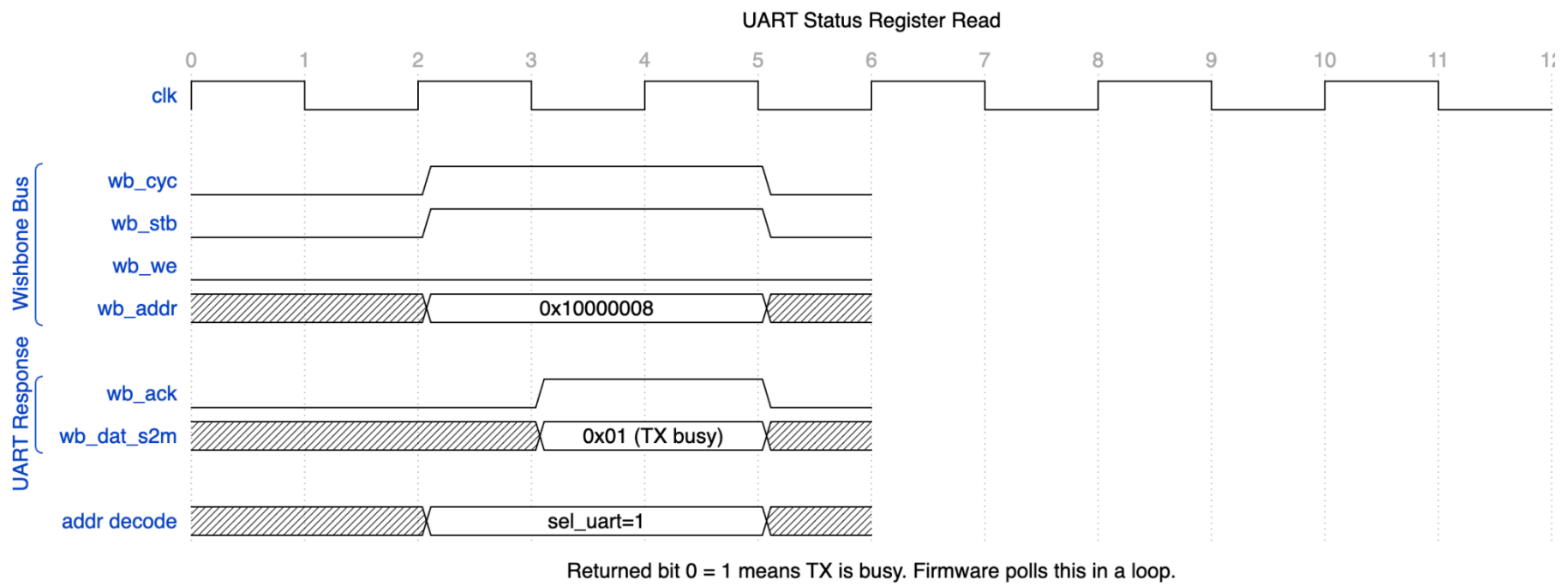


Figure 4: UART Status Register Read

Code:

```
{ signal: [
  { name: "clk",    wave: "p.....", period: 2 },
  {},
  ["Wishbone Bus",
    { name: "wb_cyc",  wave: "0.1..0" },
    { name: "wb_stb",  wave: "0.1..0" },
    { name: "wb_we",   wave: "0....." },
    { name: "wb_addr", wave: "x.=.x", data: ["0x10000008"] }
  ],
  {},
  ["UART Response",
    { name: "wb_ack",  wave: "0..1.0" },
    { name: "wb_dat_s2m", wave: "x.=.x", data: ["0x01 (TX busy)"] }
  ],
  {},
  { name: "addr decode", wave: "x.=.x", data: ["sel_uart=1"] }
],
head: {
  text: "UART Status Register Read",
  tick: 0
},
foot: {
  text: "Returned bit 0 = 1 means TX is busy. Firmware polls this in a loop."
},
config: { hscale: 2 }
}
```

Explanation:

Block diagram reference: Top-Level SoC -> CPU -> wb_interconnect -> wb_uart (STATUS reg)

Identical bus timing to BRAM read. The interconnect routes to UART based on address decode, and the UART returns its status bits.

Address decode path (from your Wishbone Interconnect block diagram):

- $\text{addr}[31:28] = 0x1$ -> not RAM, passes to peripheral decode
- $\text{addr}[15:12] = 0x0$ -> selects UART
- $\text{addr}[3:2] = 0x2$ -> offset $0x8$ = STATUS register

Identical bus timing to the BRAM read — the UART peripheral is also a single-cycle slave. The only difference is routing: the interconnect sees $\text{addr}[31:28] = 0x1$, $\text{addr}[15:12] = 0x0$ → selects UART. The offset $\text{addr}[3:2] = 0x2$ selects the STATUS register.

The returned data $0x00000001$ means bit 0 (TX busy) is set. Firmware would loop reading this register until bit 0 clears before sending the next byte.

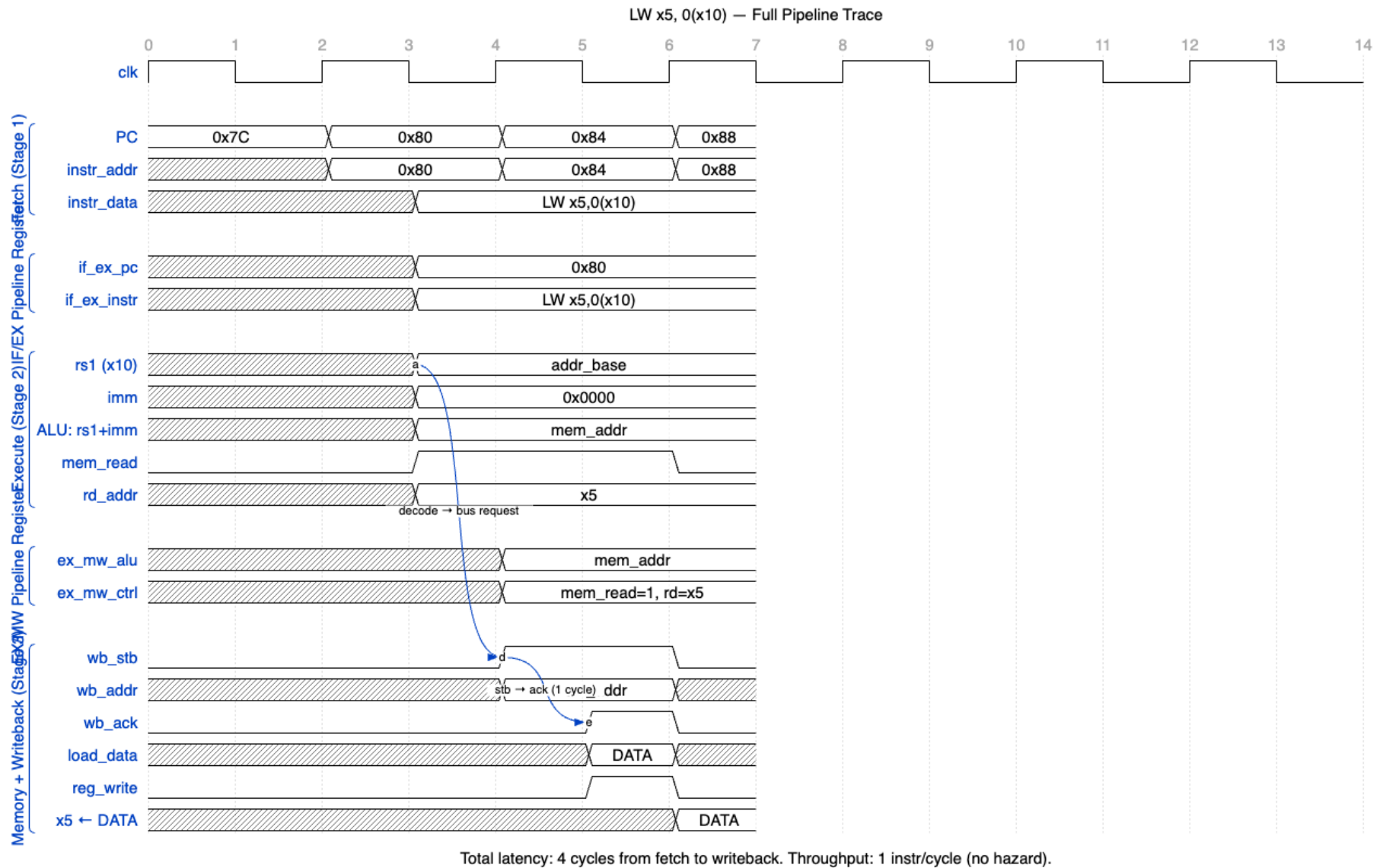


Figure 5: Full Pipeline Trace

Code:

```
{ signal: [
  { name: "clk",      wave: "p.....", period: 2 },
  {},
  ["Fetch (Stage 1)",
    { name: "PC",      wave: "=.=.=.", data: ["0x7C","0x80","0x84","0x88"] },
    { name: "instr_addr", wave: "x.=.=.", data: ["0x80","0x84","0x88"] },
    { name: "instr_data", wave: "x..=...", data: ["LW x5,0(x10)"] }
  ],
  {},
  ["IF/EX Pipeline Register",
    { name: "if_ex_pc",   wave: "x..=...", data: ["0x80"] },
    { name: "if_ex_instr", wave: "x..=...", data: ["LW x5,0(x10)"] }
  ],
  {},
  ["Execute (Stage 2)",
    { name: "rs1 (x10)",  wave: "x..=...", data: ["addr_base"], node: "...a..." },
    { name: "imm",        wave: "x..=...", data: ["0x0000"] },
    { name: "ALU: rs1+imm", wave: "x..=...", data: ["mem_addr"] },
    { name: "mem_read",   wave: "0..1..0" },
    { name: "rd_addr",    wave: "x..=...", data: ["x5"] }
  ],
  {},
  ["EX/MW Pipeline Register",
    { name: "ex_mw_alu",  wave: "x...=..", data: ["mem_addr"] },
    { name: "ex_mw_ctrl", wave: "x...=..", data: ["mem_read=1, rd=x5"] }
  ],
  {},
  ["Memory + Writeback (Stage 3)",
    { name: "wb_stb",     wave: "0...1.0", node: "....d.." },
    { name: "wb_addr",    wave: "x...=x", data: ["mem_addr"] },
    { name: "wb_ack",     wave: "0....10", node: ".....e." },
```

```

    { name: "load_data",  wave: "x....=x",  data: ["DATA"] },
    { name: "reg_write",  wave: "0....10" },
    { name: "x5 ← DATA", wave: "x.....=",  data: ["DATA"] }
  ]
],
edge: [
  "a~>d decode → bus request",
  "d~>e stb → ack (1 cycle)",
  "e~>f x5 written on rising edge"
],
head: {
  text: "LW x5, 0(x10) — Full Pipeline Trace",
  tick: 0
},
foot: {
  text: "Total latency: 4 cycles from fetch to writeback. Throughput: 1 instr/cycle (no hazard)."
},
config: { hscale: 2 }
}

```

Explanation:

Block diagram reference: CPU Pipeline -> Fetch -> IF/EX -> Execute -> EX/MW -> Mem+WB

This is the big one. Traces LW x5, 0(x10) through every pipeline stage.

Reading this diagram stage by stage:

1. Cycle 1 (Fetch): PC=0x80 goes to BRAM, instruction comes back
2. Cycle 2 (Execute): IF/EX reg holds LW. Decoder extracts rs1=x10, rd=x5, imm=0. ALU computes address = x10 + 0. mem_read goes HIGH.
3. Cycle 3 (Mem+WB): EX/MW reg passes address to Wishbone bus. stb asserted.
4. Cycle 4 (Mem+WB cont.): BRAM returns data with ack. x5 gets written.

Cycle 1 (Fetch): PC = 0x80 is sent to BRAM. BRAM returns the LW instruction on the next edge.

Cycle 2 (Execute): IF/EX register holds the instruction. Decoder outputs rs1=x10, rd=x5, imm=0, mem_read=1. ALU computes address = x10 + 0.

Cycle 3-4 (Mem+WB): The computed address is put on the Wishbone bus (stb asserted). BRAM responds with data and ack in cycle 4. On the rising edge of cycle 5, the data is written to register x5.

Note: if the next instruction needs x5, the pipeline must stall for 1 cycle a load-use hazard. See the Pipeline Hazard tab.

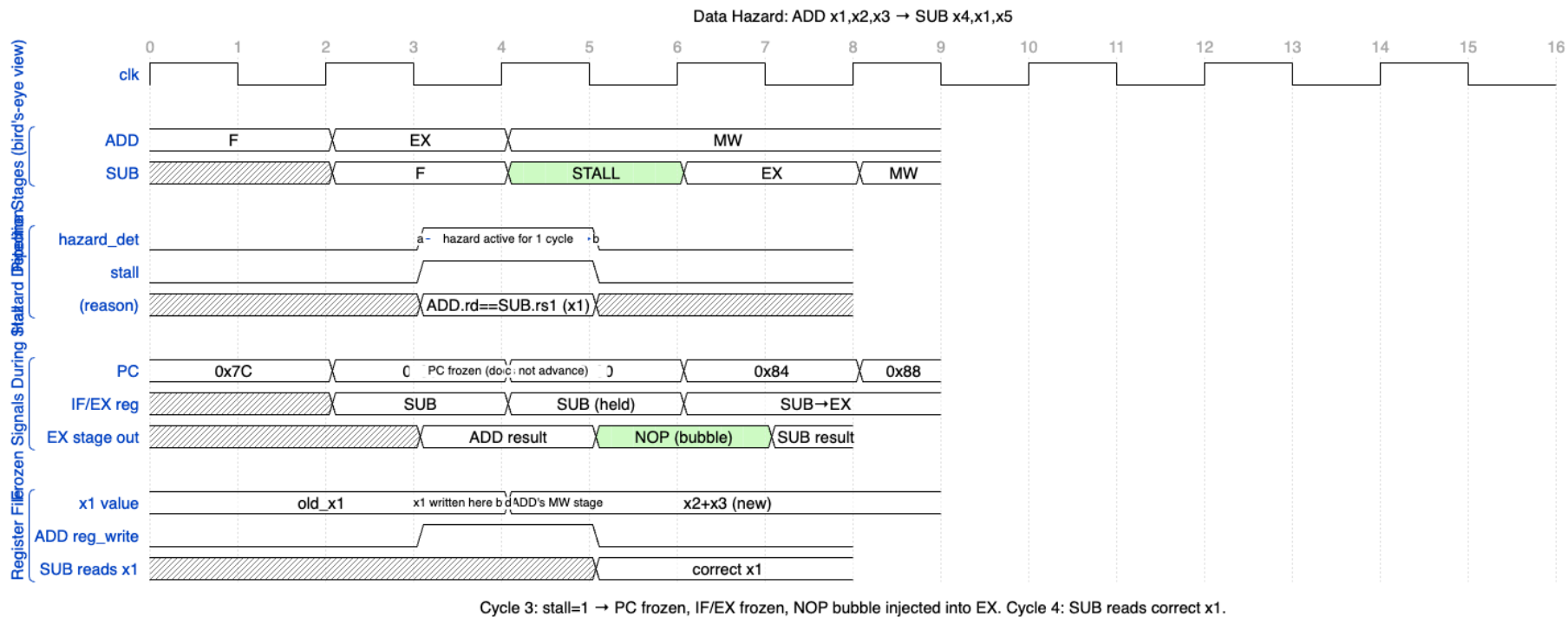


Figure 6: Data Hazard Situation

Code:

```
{ signal: [
  { name: "clk",      wave: "p.....", period: 2 },
  {},
  ["Pipeline Stages (bird's-eye view)",
    { name: "ADD",      wave: "=.=.=....", data: ["F","EX","MW"] },
    { name: "SUB",      wave: "x.=.7.==", data: ["F","STALL","EX","MW"] }
  ],
  {},
  ["Hazard Detection",
    { name: "hazard_det", wave: "0..1.0..",      node: "...a.b.." },
    { name: "stall",      wave: "0..1.0.." },
    { name: " (reason)", wave: "x..=x..", data: ["ADD.rd==SUB.rs1 (x1)"] }
  ],
  {},
  ["Frozen Signals During Stall",
    { name: "PC",      wave: "=.=.=.=.", data: ["0x7C","0x80","0x80","0x84","0x88"],
      node: "....c...." },
    { name: "IF/EX reg", wave: "x.=.=.=.", data: ["SUB","SUB (held)","SUB→EX"] },
    { name: "EX stage out", wave: "x..=7.==", data: ["ADD result","NOP (bubble)","SUB result"] }
  ],
  {},
  ["Register File",
    { name: "x1 value", wave: "=...=....", data: ["old_x1","x2+x3 (new)"],
      node: "....d...." },
    { name: "ADD reg_write", wave: "0..1.0.." },
    { name: "SUB reads x1", wave: "x....=..", data: ["correct x1"] }
  ]
],
edge: [
  "a~>b hazard active for 1 cycle",
  "c~>c PC frozen (does not advance)",
```

```
"d-~>d x1 written here by ADD's MW stage"
],
head: {
  text: "Data Hazard: ADD x1,x2,x3 → SUB x4,x1,x5",
  tick: 0
},
foot: {
  text: "Cycle 3: stall=1 → PC frozen, IF/EX frozen, NOP bubble injected into EX. Cycle 4: SUB reads correct x1."
},
config: { hscale: 2 }
}
```

Explanation:

Block diagram reference: CPU Pipeline -> Hazard Detection Unit -> Stall Logic

ADD x1, x2, x3 followed by SUB x4, x1, x5, SUB reads x1 which ADD hasn't written yet. The hazard unit inserts a 1-cycle stall (bubble).

What the stall does (three simultaneous actions):

1. PC is frozen — it does NOT advance from 0x80 to 0x84
2. IF/EX register is frozen — SUB instruction stays in place, not lost
3. A NOP bubble is injected into the Execute stage output (EX/MW gets a NOP)

Meanwhile, ADD completes its MW stage and writes $x1 = x2 + x3$. On cycle 4, SUB enters Execute and reads the correct, freshly-written value of x1.

The 7 character in WaveDrom produces a red/orange-colored data block, perfect for highlighting the stall bubble visually.

Cycle 1 (Fetch): PC = 0x80 is sent to BRAM. BRAM returns the LW instruction on the next edge.

Cycle 2 (Execute): IF/EX register holds the instruction. Decoder outputs $rs1=x10$, $rd=x5$, $imm=0$, $mem_read=1$. ALU computes $address = x10 + 0$.

Cycle 3-4 (Mem+WB): The computed address is put on the Wishbone bus (stb asserted). BRAM responds with data and ack in cycle 4. On the rising edge of cycle 5, the data is written to register x5.

Note: if the next instruction needs x5, the pipeline must stall for 1 cycle, a load-use hazard. See the Pipeline Hazard tab.

Module Interface List

Module Interface List — RISC-V SoC (Artefact 1.4)

Project: RV32I SoC on Tang Nano 20K

Derived from: Top-Level SoC diagram, CPU Pipeline diagram, Wishbone Interconnect diagram

Rule: Every box = a module. Every labelled arrow = a port. If it's not in this table, it doesn't exist.

1. rv32i_alu

Type: Combinational (no clock)

Block diagram source: CPU Pipeline → Execute stage → "ALU" box

C++ analogy: A pure function same inputs always produce same outputs, no state.

Direction	Signal	Width	Description
input	a	[31:0]	First operand (from rs1_data via reg file)
input	b	[31:0]	Second operand (from ALU B mux: rs2_data or immediate)
input	op	[3:0]	Operation select: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
output	result	[31:0]	Computation result
output	zero	1	HIGH when result == 0 (used by branch logic)
output	negative	1	HIGH when result[31] == 1 (signed negative)
output	overflow	1	HIGH on signed overflow

Notes:

- 10 operations encoded in 4-bit op field
- No clock input, this is entirely combinational logic (like always @(*))
- Zero flag feeds branch logic for BEQ/BNE

2. rv32i_regfile

Type: Sequential (synchronous write, asynchronous read)

Block diagram source: CPU Pipeline → Execute stage → "Register File" box

C++ analogy: An array of 32 ints with 2 read ports and 1 write port: int32_t regs[32]

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Synchronous reset
input	rs1_addr	[4:0]	Read port 1 address (from decoder rs1 field)
input	rs2_addr	[4:0]	Read port 2 address (from decoder rs2 field)
input	rd_addr	[4:0]	Write port address (from EX/MW pipeline register rd field)
input	rd_data	[31:0]	Write data (ALU result, load data, or PC+4 depending on instruction)
input	rd_we	1	Write enable (from EX/MW reg_write control signal)
output	rs1_data	[31:0]	Read port 1 data → feeds ALU input A and branch logic
output	rs2_data	[31:0]	Read port 2 data → feeds ALU B mux and store data path

Notes:

- x0 is hardwired to zero: reads always return 0, writes are silently discarded
- Reads are asynchronous (combinational) , data available same cycle
- Writes are synchronous , happen on rising clock edge
- Write port is driven from the Memory/Writeback stage (stage 3)

3. rv32i_decoder

Type: Combinational (no clock)

Block diagram source: CPU Pipeline → Execute stage → "Decoder" + "Immediate Generator" boxes

C++ analogy: A giant switch statement on the opcode that extracts fields and sets control flags.

Direction	Signal	Width	Description
input	instr	[31:0]	Raw 32-bit instruction word (from IF/EX pipeline register)
output	rs1	[4:0]	Source register 1 address (instr[19:15])
output	rs2	[4:0]	Source register 2 address (instr[24:20])
output	rd	[4:0]	Destination register address (instr[11:7])
output	imm	[31:0]	Sign-extended immediate (format depends on instruction type)

output	alu_op	[3:0]	ALU operation code
output	alu_src	1	ALU B input select: 0 = rs2_data, 1 = immediate
output	mem_read	1	HIGH for load instructions (LB, LH, LW, LBU, LHU)
output	mem_write	1	HIGH for store instructions (SB, SH, SW)
output	reg_write	1	HIGH if instruction writes to rd
output	mem_to_reg	1	Writeback mux: 0 = ALU result, 1 = memory load data
output	branch	1	HIGH for B-type instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU)
output	jump	1	HIGH for JAL/JALR
output	lui	1	HIGH for LUI (load upper immediate)
output	auipc	1	HIGH for AUIPC (add upper immediate to PC)
output	funct3	[2:0]	Passed through for branch type selection and load/store size

Notes:

- Immediate generator handles all 6 formats: I, S, B, U, J (R-type has no immediate)
- B-type and J-type immediates have scrambled bits that must be reassembled
- Sign bit is always instr[31] across all formats
- funct3 is needed downstream for branch comparisons (BEQ vs BNE vs BLT etc.) and load/store width (byte/half/word)

4. rv32i_hazard

Type: Combinational (no clock)

Block diagram source: CPU Pipeline → "Hazard Detection Unit" box (below pipeline)

C++ analogy: An if-statement that checks whether two threads are fighting over the same variable.

Direction	Signal	Width	Description
input	ex_mw_rd	[4:0]	Destination register of instruction in MW stage
input	ex_mw_reg_write	1	Whether MW stage instruction writes to a register
input	ex_mw_mem_read	1	Whether MW stage instruction is a load (load-use hazard)

input	decode_rs1	[4:0]	Source register 1 of instruction being decoded (EX stage)
input	decode_rs2	[4:0]	Source register 2 of instruction being decoded
input	branch_taken	1	Branch resolved as taken in EX stage
output	stall	1	Freezes PC and IF/EX register for 1 cycle
output	flush_ifex	1	Replaces IF/EX register contents with NOP (branch flush)
output	flush_exmw	1	Replaces EX/MW register contents with NOP (data hazard bubble)

Notes:

- Data hazard: stall when `ex_mw_rd != 0 AND ex_mw_reg_write AND (ex_mw_rd == decode_rs1 OR ex_mw_rd == decode_rs2)`
- Control hazard: `flush_ifex` when `branch_taken` (the fetched instruction after the branch is wrong)
- Stall freezes PC + IF/EX register simultaneously, injects NOP bubble into EX/MW

5. rv32i_pipeline_regs

Type: Sequential (clocked flip-flops)

Block diagram source: CPU Pipeline → "IF/EX Register" and "EX/MW Register" boxes

C++ analogy: Two structs that get copied forward on each clock tick: `struct IFEX { ... }; struct EXMW { ... };`

IF/EX Register

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Synchronous reset
input	stall	1	When HIGH, register holds its value (does not update)
input	flush	1	When HIGH, register clears to NOP
input	pc_in	[31:0]	PC of the fetched instruction
input	instr_in	[31:0]	The raw instruction word from BRAM
output	pc_out	[31:0]	Latched PC → feeds Execute stage

output	instr_out	[31:0]	Latched instruction → feeds Decoder
--------	-----------	--------	-------------------------------------

EX/MW Register

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Synchronous reset
input	flush	1	When HIGH, register clears to NOP (data hazard bubble)
input	alu_result_in	[31:0]	ALU computation result
input	rs2_data_in	[31:0]	Store data (rs2 value, passed through for SW/SH/SB)
input	rd_in	[4:0]	Destination register address
input	pc_plus4_in	[31:0]	PC+4 for JAL/JALR return address writeback
input	reg_write_in	1	Control: write to register file
input	mem_read_in	1	Control: this is a load instruction
input	mem_write_in	1	Control: this is a store instruction
input	mem_to_reg_in	1	Control: writeback source (0=ALU, 1=memory)
input	wb_we_in	1	Wishbone write enable
input	wb_sel_in	[3:0]	Wishbone byte select (byte/half/word)
input	funct3_in	[2:0]	For load sign/zero extension (LB vs LBU etc.)
output	(all corresponding _out signals)	—	Same signals, latched by one clock cycle

6. rv32i_core (top-level CPU)

Type: Sequential (contains all pipeline stages)

Block diagram source: Entire CPU Pipeline diagram; also the "CPU" box on the Top-Level SoC diagram

C++ analogy: The main class that instantiates ALU, reg file, decoder, hazard unit, and pipeline registers.

Direction	Signal	Width	Description
input	clk	1	System clock (27 MHz)
input	rst	1	Synchronous reset
input	wb_dat_s2m	[31:0]	Read data from Wishbone bus (slave → master)
input	wb_ack	1	Acknowledge from Wishbone bus
output	wb_cyc	1	Bus cycle active
output	wb_stb	1	Strobe: valid transfer in progress
output	wb_we	1	Write enable (1=write, 0=read)
output	wb_addr	[31:0]	Address (instruction fetch or data load/store)
output	wb_dat_m2s	[31:0]	Write data (master → slave, for stores)
output	wb_sel	[3:0]	Byte select: 4'b1111=word, 4'b0011=half, 4'b0001=byte

Notes:

- This is the Wishbone master — the only thing on the bus that initiates transactions
- Instruction fetch and data load/store share this single bus port (the interconnect routes both)
- Internal submodules (ALU, reg file, etc.) are NOT visible at this level — they're internal wiring

7. wb_interconnect

Type: Combinational (address decode + mux, no state)

Block diagram source: Wishbone Interconnect diagram (Address Decode, STB Gating, Read MUX/Ack)

C++ analogy: A router/switch statement: looks at the address, forwards the request to the right peripheral.

From CPU master (inputs)

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Reset

input	cpu_cyc	1	Bus cycle from CPU
input	cpu_stb	1	Strobe from CPU
input	cpu_we	1	Write enable from CPU
input	cpu_addr	[31:0]	Address from CPU
input	cpu_dat_m2s	[31:0]	Write data from CPU
input	cpu_sel	[3:0]	Byte select from CPU

From each slave (inputs) 5 sets

Direction	Signal	Width	Description
input	ram_dat_o	[31:0]	Read data from BRAM
input	ram_ack	1	Acknowledge from BRAM
input	uart_dat_o	[31:0]	Read data from UART
input	uart_ack	1	Acknowledge from UART
input	gpio_dat_o	[31:0]	Read data from GPIO
input	gpio_ack	1	Acknowledge from GPIO
input	pwm_dat_o	[31:0]	Read data from PWM
input	pwm_ack	1	Acknowledge from PWM
input	timer_dat_o	[31:0]	Read data from Timer
input	timer_ack	1	Acknowledge from Timer

Back to CPU master (outputs)

Direction	Signal	Width	Description
output	cpu_dat_s2m	[31:0]	Muxed read data back to CPU
output	cpu_ack	1	Muxed acknowledge back to CPU

To each slave (outputs) — shared bus + individual strobes

Direction	Signal	Width	Description
output	slave_addr	[31:0]	Address (shared, directly from cpu_addr)
output	slave_dat_i	[31:0]	Write data (shared, directly from cpu_dat_m2s)
output	slave_we	1	Write enable (shared, directly from cpu_we)
output	slave_sel	[3:0]	Byte select (shared, directly from cpu_sel)
output	ram_stb	1	Strobe gated to BRAM: cpu_stb & cpu_cyc & sel_ram
output	uart_stb	1	Strobe gated to UART: cpu_stb & cpu_cyc & sel_uart
output	gpio_stb	1	Strobe gated to GPIO: cpu_stb & cpu_cyc & sel_gpio
output	pwm_stb	1	Strobe gated to PWM: cpu_stb & cpu_cyc & sel_pwm
output	timer_stb	1	Strobe gated to Timer: cpu_stb & cpu_cyc & sel_timer

Address decode logic (internal, not ports):

```

sel_ram  = (cpu_addr[31:28] == 4'h0)
sel_uart = (cpu_addr[31:28] == 4'h1) && (cpu_addr[15:12] == 4'h0)
sel_gpio = (cpu_addr[31:28] == 4'h1) && (cpu_addr[15:12] == 4'h1)
sel_pwm  = (cpu_addr[31:28] == 4'h1) && (cpu_addr[15:12] == 4'h2)
sel_timer = (cpu_addr[31:28] == 4'h1) && (cpu_addr[15:12] == 4'h3)

```

8. wb_bram

Type: Sequential (synchronous BRAM)

Block diagram source: Top-Level SoC → "BRAM" box

C++ analogy: `std::array<uint32_t, 8192>` memory; with single-cycle read/write.

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Reset
input	wb_stb	1	Strobe (from interconnect ram_stb)
input	wb_cyc	1	Bus cycle
input	wb_we	1	Write enable

input	wb_addr	[31:0]	Address (only bits [14:2] used for 32KB word-addressed)
input	wb_dat_i	[31:0]	Write data
input	wb_sel	[3:0]	Byte select
output	wb_dat_o	[31:0]	Read data
output	wb_ack	1	Acknowledge (asserted 1 cycle after stb)

Notes:

- 32KB = 8192 words $\times$ 32 bits
- addr[14:2] selects the word (lower 2 bits ignored — word-aligned)
- Initialised from firmware hex file via \$readmemh("firmware.hex", mem)
- Single-cycle slave: ack returns 1 clock after stb

9. wb_uart

Type: Sequential

Block diagram source: Top-Level SoC $\rightarrow$ "UART" box (with UART_TX output, UART_RX input)

C++ analogy: A serial port class with TX shift register, RX shift register, and status flags.

Direction	Signal	Width	Description
input	clk	1	System clock (27 MHz)
input	rst	1	Reset
input	wb_stb	1	Strobe (from interconnect uart_stb)
input	wb_we	1	Write enable
input	wb_addr	[3:0]	Register select: 0x0=TX_DATA, 0x4=RX_DATA, 0x8=STATUS, 0xC=CONTROL
input	wb_dat_i	[31:0]	Write data
input	uart_rx	1	Physical UART receive pin (from FPGA pad)
output	wb_dat_o	[31:0]	Read data (register contents)
output	wb_ack	1	Acknowledge

output	uart_tx	1	Physical UART transmit pin (to FPGA pad)
--------	---------	---	--

Register map (addr[3:2]):

- 0x0: TX_DATA — write byte to transmit
- 0x4: RX_DATA — read received byte
- 0x8: STATUS — bit 0 = TX busy, bit 1 = RX valid
- 0xC: CONTROL — bit 0 = RX interrupt enable

Baud rate: $27,000,000 / 115,200 = 234$ cycles per bit. 10 bits per frame = 2340 cycles.

10. wb_gpio

Type: Sequential

Block diagram source: Top-Level SoC → "GPIO" box (with gpio_out[7:0] output, gpio_in[7:0] input)

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Reset
input	wb_stb	1	Strobe (from interconnect gpio_stb)
input	wb_we	1	Write enable
input	wb_addr	[3:0]	Register select: 0x0=DIR, 0x4=OUT, 0x8=IN
input	wb_dat_i	[31:0]	Write data
input	gpio_in	[7:0]	Physical input pins (buttons, switches)
output	wb_dat_o	[31:0]	Read data
output	wb_ack	1	Acknowledge
output	gpio_out	[7:0]	Physical output pins (LEDs)

Register map (addr[3:2]):

- 0x0: DIR — direction register (1 = output per pin)
- 0x4: OUT — output value register
- 0x8: IN — input value register (read-only)

11. wb_pwm

Type: Sequential

Block diagram source: Top-Level SoC → "PWM" box (with pwm_out output)

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Reset
input	wb_stb	1	Strobe (from interconnect pwm_stb)
input	wb_we	1	Write enable
input	wb_addr	[3:0]	Register select: 0x0=PERIOD, 0x4=DUTY, 0x8=CONTROL
input	wb_dat_i	[31:0]	Write data
output	wb_dat_o	[31:0]	Read data
output	wb_ack	1	Acknowledge
output	pwm_out	1	Physical PWM output pin

Register map (addr[3:2]):

- 0x0: PERIOD — counter top value
- 0x4: DUTY — compare value (output HIGH when counter < DUTY)
- 0x8: CONTROL — bit 0 = enable

12. wb_timer

Type: Sequential

Block diagram source: Top-Level SoC → "Timer" box (with timer_irq output)

Direction	Signal	Width	Description
input	clk	1	System clock
input	rst	1	Reset
input	wb_stb	1	Strobe (from interconnect timer_stb)
input	wb_we	1	Write enable

input	wb_addr	[3:0]	Register select: 0x0=COUNT, 0x4=COMPARE, 0x8=CONTROL, 0xC=PRESCALER
input	wb_dat_i	[31:0]	Write data
output	wb_dat_o	[31:0]	Read data
output	wb_ack	1	Acknowledge
output	timer_irq	1	Interrupt request (HIGH when COUNT reaches COMPARE)

Register map (addr[3:2]):

- 0x0: COUNT — current counter value (read-only)
 - 0x4: COMPARE — value that triggers interrupt
 - 0x8: CONTROL — bit 0 = enable, bit 1 = interrupt enable
 - 0xC: PRESCALER — clock divider value
-

13. soc_top (top-level, instantiates everything)

Type: Sequential

Block diagram source: Top-Level SoC diagram — the outer boundary

These are the physical FPGA pins.

Direction	Signal	Width	Description
input	clk	1	27 MHz crystal oscillator (pin 4)
input	rst_n	1	Active-low reset button (pin 88)
input	uart_rx	1	UART receive pin (pin 18)
output	uart_tx	1	UART transmit pin (pin 17)
output	led	[7:0]	LEDs (directly from gpio_out)
output	pwm_out	1	PWM output pin

Notes:

- rst_n is active-low (button pressed = 0). Invert it internally to create active-high rst.
- All internal modules connect through the wb_interconnect.
- The constraint file (.cst) maps these signal names to physical FPGA pin numbers.

Cross-Reference Checklist

Use this to verify every connection. For each row, confirm both sides of the wire agree on name and width.

From module	Signal	→	To module	Signal
rv32i_core	wb_addr[31:0]	→	wb_interconnect	cpu_addr[31:0]
rv32i_core	wb_dat_m2s[31:0]	→	wb_interconnect	cpu_dat_m2s[31:0]
rv32i_core	wb_cyc	→	wb_interconnect	cpu_cyc
rv32i_core	wb_stb	→	wb_interconnect	cpu_stb
rv32i_core	wb_we	→	wb_interconnect	cpu_we
rv32i_core	wb_sel[3:0]	→	wb_interconnect	cpu_sel[3:0]
wb_interconnect	cpu_dat_s2m[31:0]	→	rv32i_core	wb_dat_s2m[31:0]
wb_interconnect	cpu_ack	→	rv32i_core	wb_ack
wb_interconnect	ram_stb	→	wb_bram	wb_stb
wb_interconnect	uart_stb	→	wb_uart	wb_stb
wb_interconnect	gpio_stb	→	wb_gpio	wb_stb
wb_interconnect	pwm_stb	→	wb_pwm	wb_stb
wb_interconnect	timer_stb	→	wb_timer	wb_stb
wb_bram	wb_dat_o[31:0]	→	wb_interconnect	ram_dat_o[31:0]
wb_bram	wb_ack	→	wb_interconnect	ram_ack
wb_uart	wb_dat_o[31:0]	→	wb_interconnect	uart_dat_o[31:0]
wb_uart	wb_ack	→	wb_interconnect	uart_ack
wb_gpio	wb_dat_o[31:0]	→	wb_interconnect	gpio_dat_o[31:0]
wb_gpio	wb_ack	→	wb_interconnect	gpio_ack
wb_pwm	wb_dat_o[31:0]	→	wb_interconnect	pwm_dat_o[31:0]
wb_pwm	wb_ack	→	wb_interconnect	pwm_ack

wb_timer	wb_dat_o[31:0]	→	wb_interconnect	timer_dat_o[31:0]
wb_timer	wb_ack	→	wb_interconnect	timer_ack
soc_top	uart_rx	→	wb_uart	uart_rx
wb_uart	uart_tx	→	soc_top	uart_tx
soc_top (external pins)	gpio_in[7:0]	→	wb_gpio	gpio_in[7:0]
wb_gpio	gpio_out[7:0]	→	soc_top	led[7:0]
wb_pwm	pwm_out	→	soc_top	pwm_out
wb_timer	timer_irq	→	rv32i_core	(future: interrupt input)
decoder	rs1[4:0]	→	regfile	rs1_addr[4:0]
decoder	rs2[4:0]	→	regfile	rs2_addr[4:0]
decoder	alu_op[3:0]	→	ALU	op[3:0]
decoder	imm[31:0]	→	ALU B mux	(via alu_src select)
regfile	rs1_data[31:0]	→	ALU	a[31:0]
regfile	rs2_data[31:0]	→	ALU B mux / store path	
ALU	result[31:0]	→	EX/MW register	alu_result_in[31:0]
ALU	zero	→	branch logic	(branch decision)
hazard	stall	→	PC + IF/EX register	(freeze)
hazard	flush_ifex	→	IF/EX register	flush
hazard	flush_exmw	→	EX/MW register	flush

Test Plan

Test Plan — RISC-V SoC (Artefact 1.5)

Rule: Every test is written here BEFORE writing code. Each test has concrete inputs and expected outputs.

Tool: cocotb (Python) + Icarus Verilog. Waveform dumps via VCD → GTKWave for debugging.

Pass criteria listed per module — don't move to the next module until all tests pass.

1. rv32i_alu

Pass gate: All 10 operations correct, including every edge case below.

Test type: Combinational — no clock needed. Drive inputs, wait 1ns, check outputs.

#	Test	Inputs	Expected Output	Why this matters
1.1	ADD basic	a=5, b=3, op=ADD	result=8, zero=0	Sanity check
1.2	ADD overflow wrap	a=0xFFFFFFFF, b=1, op=ADD	result=0x00000000, zero=1	Unsigned overflow wraps, zero flag sets
1.3	ADD signed overflow	a=0x7FFFFFFF, b=1, op=ADD	result=0x80000000, overflow=1	Max positive + 1 = negative
1.4	SUB basic	a=10, b=3, op=SUB	result=7	
1.5	SUB underflow	a=0, b=1, op=SUB	result=0xFFFFFFFF, negative=1	0 - 1 wraps to -1
1.6	AND	a=0xFF00FF00, b=0x0F0F0F0F, op=AND	result=0x0F000F00	
1.7	OR	a=0xFF00FF00, b=0x0F0F0F0F, op=OR	result=0xFF0FFF0F	
1.8	XOR	a=0xAAAAAAAA, b=0x55555555, op=XOR	result=0xFFFFFFFF	All bits flip
1.9	SLL basic	a=1, b=4, op=SLL	result=16	Shift left by 4

1.1 0	SLL by 0	a=0xDEADBEEF, b=0, op=SLL	result=0xDEADBEEF	Shift by 0 = no change
1.11	SLL by 31	a=1, b=31, op=SLL	result=0x80000000	Max shift
1.1 2	SRL basic	a=0x80000000, b=4, op=SRL	result=0x08000000	Logical: fills with 0
1.1 3	SRA positive	a=0x40000000, b=4, op=SRA	result=0x04000000	Positive number: same as SRL
1.1 4	SRA negative	a=0x80000000, b=4, op=SRA	result=0xF8000000	Negative: fills with 1 (SRL vs SRA gotcha)
1.1 5	SLT true	a=0xFFFFFFF B (-5), b=3, op=SLT	result=1	Signed: -5 < 3
1.1 6	SLT false	a=3, b=0xFFFFFFF B, op=SLT	result=0	Signed: 3 > -5
1.1 7	SLTU true	a=3, b=0xFFFFFFFF, op=SLTU	result=1	Unsigned: 3 < 4294967295
1.1 8	SLTU false	a=0xFFFFFFFF, b=3, op=SLTU	result=0	Unsigned: -1 is HUGE, not small (SLT vs SLTU gotcha)
1.1 9	ADD zero result	a=5, b=0xFFFFFFF B (-5), op=ADD	result=0, zero=1	Zero flag must trigger

2. rv32i_regfile

Pass gate: x0 always zero. Simultaneous reads correct. Write-then-read correct.

Test type: Sequential — needs clock for writes, combinational for reads.

#	Test	Action	Expected	Why
2. 1	Read x0 at reset	Read rs1_addr=0	rs1_data=0	x0 must be 0 after reset

2. 2	Write x0, read back	Write 0xDEADBEEF to rd_addr=0, then read rs1_addr=0	rs1_data=0	x0 ignores all writes
2. 3	Write x1, read back	Write 42 to rd_addr=1, then read rs1_addr=1	rs1_data=42	Basic write/read
2. 4	Two simultaneous reads	Write 10 to x1, write 20 to x2. Read rs1_addr=1, rs2_addr=2 same cycle	rs1_data=10, rs2_data=20	Both read ports work simultaneously
2. 5	Write-then-read same cycle	Write 99 to x5 and read rs1_addr=5 on same clock edge	rs1_data=99 (with bypass) or old value (without)	Determines if you need a bypass mux
2. 6	All 31 registers	Write unique values to x1-x31, read each back	All match	Full register coverage
2. 7	Reset clears all	Write values to several regs, assert rst, read them back	All return 0	Reset works

3. rv32i_decoder

Pass gate: All 6 instruction formats decode correctly. All immediates sign-extended correctly.

Test type: Combinational — feed instruction word, check all output fields.

#	Test	Input (instr hex)	Key outputs to check	Why
3.1	R-type ADD	ADD x1,x2,x3	rs1=2, rs2=3, rd=1, alu_op=ADD, reg_write=1, alu_src=0	Basic R-type
3.2	R-type SUB	SUB x1,x2,x3	alu_op=SUB	funct7 bit differentiates ADD/SUB
3.3	I-type ADDI positive	ADDI x1,x2,5	rs1=2, rd=1, imm=5, alu_src=1	Positive immediate
3.4	I-type ADDI negative	ADDI x1,x2,-5	imm=0xFFFFFFFFB (sign-extended)	Sign extension - most common bug

3.5	S-type SW	SW x3,0(x2)	rs1=2, rs2=3, imm=0, mem_write=1	S-type immediate split
3.6	S-type offset	SW x3,36(x2)	imm=36	S-type immediate reassembly
3.7	B-type BEQ	BEQ x2,x3,offset	branch=1, funct3=000	Branch decode
3.8	B-type negative offset	BEQ with negative offset	imm = sign-extended negative	B-type bit scrambling + sign ext
3.9	U-type LUI	LUI x1,0x12345	rd=1, imm=0x12345000, lui=1	Upper 20 bits, lower 12 = zero
3.10	U-type AUIPC	AUIPC x1,0x12345	auipc=1	Same format as LUI, different opcode
3.11	J-type JAL	JAL x1,8	rd=1, imm=8, jump=1	J-type bit scrambling
3.12	J-type negative	JAL with negative offset	imm = sign-extended negative	J-type sign extension
3.13	I-type LW	LW x1,0(x2)	mem_read=1, mem_to_reg=1, alu_src=1	Load control signals
3.14	NOP	0x00000013	All control signals benign, rd=0	Canonical NOP
3.15	JALR	JALR x1,0(x1)	jump=1, alu_src=1	JALR uses rs1+imm, not PC+imm

4. rv32i_hazard

Pass gate: Stalls and flushes trigger in all correct scenarios and ONLY correct scenarios.

#	Test	Inputs	Expected	Why
---	------	--------	----------	-----

4.1	No hazard	ex_mw_rd=1, decode_rs1=2, decode_rs2=3	stall=0	No register overlap
4.2	RAW hazard on rs1	ex_mw_rd=5, reg_write=1, decode_rs1=5	stall=1, flush_exmw=1	Classic data hazard
4.3	RAW hazard on rs2	ex_mw_rd=5, reg_write=1, decode_rs2=5	stall=1, flush_exmw=1	Hazard on second source
4.4	x0 - NO stall	ex_mw_rd=0, reg_write=1, decode_rs1=0	stall=0	x0 always 0, no real dependency
4.5	No reg_write	ex_mw_rd=5, reg_write=0, decode_rs1=5	stall=0	MW isn't writing, no conflict
4.6	Branch taken	branch_taken=1	flush_ifex=1	Fetches instruction is wrong
4.7	Branch not taken	branch_taken=0	flush_ifex=0	Normal flow

5. rv32i_core (CPU integration)

Pass gate: All 40 RV32I instructions correct. riscv-tests compliance suite passes.

Individual instruction tests (tiny assembly programs):

#	Test	Assembly	Check
5.1	ADDI	ADDI x1, x0, 42	x1 == 42
5.2	ADD chain	ADDI x1,x0,5; ADDI x2,x0,3; ADD x3,x1,x2	x3 == 8
5.3	Negative imm	ADDI x1, x0, -1	x1 == 0xFFFFFFFF
5.4	LUI	LUI x1, 0x12345	x1 == 0x12345000
5.5	AUIPC	AUIPC x1, 0	x1 == PC of AUIPC
5.6	LW/SW round-trip	ADDI x1,x0,99; SW x1,0(x0); LW x2,0(x0)	x2 == 99

5.7	BEQ taken	x1==x2, BEQ skips one instr	Skipped instruction has no effect
5.8	BEQ not taken	x1!=x2, BEQ falls through	Next instruction executes
5.9	JAL	JAL x1, +8	x1 == return addr, PC jumps forward
5.1 0	JALR	JALR x2, x1, 0	x2 == return addr, PC == x1

Pipeline-specific tests:

#	Test	Assembly	Check
5.11	Data hazard stall	ADDI x1,x0,10; ADD x2,x1,x1	x2 == 20 (stall fires, correct value)
5.1 2	No stall needed	ADDI x1,x0,10; NOP; ADD x2,x1,x1	x2 == 20 (no stall, NOP separates)
5.1 3	Load-use hazard	LW x1,0(x0); ADD x2,x1,x1	x2 == 2*mem[0]
5.1 4	Branch flush	BEQ x0,x0,+8; ADDI x1,x0,99; ...	x1 == 0 (flushed, never executed)
5.1 5	All 6 branches	BEQ/BNE/BLT/BGE/BLTU/BGEU taken + not taken	All branch correctly
5.1 6	riscv-tests suite	Full rv32ui-p-* compliance tests	All pass

6. wb_bram

Pass gate: Single-cycle read/write. ack timing matches timing diagrams.

#	Test	Action	Expected
6.1	Write then read	Write 0xDEADBEEF to addr 0x100, read back	dat_o == 0xDEADBEEF

6.2	ack timing	Assert stb, check ack	ack HIGH exactly 1 cycle after stb
6.3	Byte select	Write with sel=4'b0001, read back	Only lowest byte changed
6.4	Multiple addresses	Write 10 different addresses, read all	All match
6.5	Hex file init	Load firmware.hex, read addr 0	Matches first word of hex

7. wb_uart

Pass gate: TX timing matches 115200 baud (234 cycles/bit). RX receives correctly.

#	Test	Action	Expected
7.1	TX byte	Write 0x41 to TX_DATA	uart_tx: start(0) + 8 data bits (LSB first) + stop(1), 234 clks/bit
7.2	TX busy flag	Write byte, read STATUS immediately	bit 0 == 1
7.3	TX done	Write byte, wait 2340+ cycles, read STATUS	bit 0 == 0
7.4	Back-to-back TX	Write byte, wait done, write another	Second byte correct after first stop bit
7.5	RX byte	Drive uart_rx with 0x55 at 115200	RX_DATA == 0x55, STATUS bit 1 == 1
7.6	Loopback	Connect tx to rx, transmit	RX_DATA matches TX_DATA

8. wb_gpio

#	Test	Action	Expected
8.1	Output	DIR=0xFF, OUT=0xAA	gpio_out == 0xAA
8.2	Input	DIR=0x00, drive gpio_in=0x55, read IN	dat_o == 0x55

8.3	Mixed	DIR=0x0F	Lower 4 driven by OUT, upper 4 readable from IN
8.4	Dir masking	DIR=0x00, OUT=0xFF	gpio_out == 0x00 (masked)

9. wb_pwm

#	Test	Action	Expected
9.1	50% duty	PERIOD=100, DUTY=50, enable	HIGH 50 clks, LOW 50 clks
9.2	0% duty	DUTY=0	Always LOW
9.3	100% duty	DUTY=PERIOD	Always HIGH
9.4	Disabled	enable=0	Always LOW

10. wb_timer

#	Test	Action	Expected
10.1	Count	Enable, read COUNT multiple times	Incrementing
10.2	Compare IRQ	COMPARE=100, enable + irq_enable	timer_irq HIGH at COUNT==100
10.3	Prescaler	PRESCALER=10	COUNT increments every 10 clocks
10.4	IRQ gating	COMPARE=50, irq_enable=0	timer_irq stays LOW

11. wb_interconnect

#	Test	Action	Expected
11.1	BRAM route	Read 0x00000100	ram_stb asserts, others LOW

11.2	UART route	Read 0x10000008	uart_stb asserts
11.3	GPIO route	Read 0x10001004	gpio_stb asserts
11.4	PWM route	Read 0x10002000	pwm_stb asserts
11.5	Timer route	Read 0x10003000	timer_stb asserts
11.6	Invalid addr	Read 0x20000000	No stb asserts
11.7	Data mux	Different dat_o per slave, read each	cpu_dat_s2m matches selected slave

12. soc_top (full system simulation)

#	Test	Action	Expected
12.1	UART boot	Load firmware printing "RISC-V SoC booted!"	All chars on uart_tx
12.2	GPIO blink	Firmware toggles gpio_out[0]	Toggles in waveform
12.3	PWM	Firmware configures 50% PWM	pwm_out correct in waveform
12.4	Timer poll	Firmware polls timer, branches at threshold	Branch at correct cycle

13. Hardware (Tang Nano 20K)

#	Test	Action	Expected
13.1	UART hello	Serial terminal 115200 8N1, reset	"RISC-V SoC booted!" appears
13.2	LED blink	Observe board	LED toggles ~0.5s
13.3	PWM brightness	PWM to LED	Visibly dimmer than full on

Debug workflow

1. Test fails → open waves.vcd in GTKWave
2. Find where actual differs from expected
3. Trace backward to find the cause
4. Fix Verilog, re-run
5. ALL tests for current module must pass before moving on